

소규모 언어 모델,
DeepSeek / Llama

소규모 언어 모델, DeepSeek / Llama

1. 딥시크 모델
2. 랭체인에서 딥시크 모델 사용하기
3. 로컬에서 라마와 임베딩 모델 사용하기

1. 딥시크 모델

- LLM을 사용하려면 질의할 때마다 토큰 크기에 따른 비용이 발생하고, 내 컴퓨터 안에 저장된 자료를, 서비스 하는 서버에 전송해야 한다는 보안 문제가 있었음
- 회사 내의 문서를 이용해 RAG를 개발하려는 경우에도 해당 문서를 오픈AI에 전송하게 되어 있어서 언어 모델을 사용할 수 없는 기업이 많았음
- 이런 문제들을 해결하기 위해 일반 PC나 스마트폰에서 실행할 수 있는 소규모 언어 모델(Small Language Model, SLM)의 수요가 늘 있어 왔음
- 이에 맞춰 대규모 언어 모델(LMM)을 저사양 PC에서도 작동할 수 있도록 성능을 다소 포기하고 경량화한 모델들이 등장함

1. 딥시크 모델

● 딥시크-R1 모델

- 2025년 초 GPT의 성능에 필적할 만한 딥시크-R1(DeepSeek-R1) 모델을 누구나 사용할 수 있는 MIT 라이선스로 공개하며 전 세계에 충격을 안김
- 대형 언어 모델은 글로벌 IT 대기업이 아니면 개발할 수 없다는 편견을 깬
- 게다가 자신들이 개발한 여러 버전의 언어 모델을 오픈소스로 공개함
- 컴퓨터 사양이 충분하다면 딥시크-R1 모델을 자신의 컴퓨터에 내려 받아 실행 수 있음
- 대규모 언어 모델을 경량화한 모델들도 함께 공개함
- 이 모델들도 기존의 소규모 언어 모델에서는 기대하기 어려웠던 수준의 답변을 내놓음
- 오픈AI가 GPT 모델을 이용해 챗GPT 웹 사이트를 운영하고 API를 서비스하는 것처럼, 딥시크 모델도 챗GPT와 같이 채팅할 수 있는 웹 사이트와 API 서비스를 제공함
- 그러나 딥시크의 채팅 웹 사이트에서 사용자의 정보를 지나치게 수집하는 것으로 알려져 있음
- 이 웹 사이트에 접근할 수 없고 많은 국가와 기관에서 딥시크 사이트 접속을 금지하고 있음
- 이번 실습하는 내용은 문제가 있던 웹 사이트나 API를 이용하는 것이 아니라, 오픈소스 모델 자체를 내려 받아 사용
- 이 방법을 활용하면 API 비용 부담 없이 내 컴퓨터 안에서 외부에 자료를 전송하지 않고도 딥시크 언어 모델을 사용하는 시스템을 구축할 수 있음

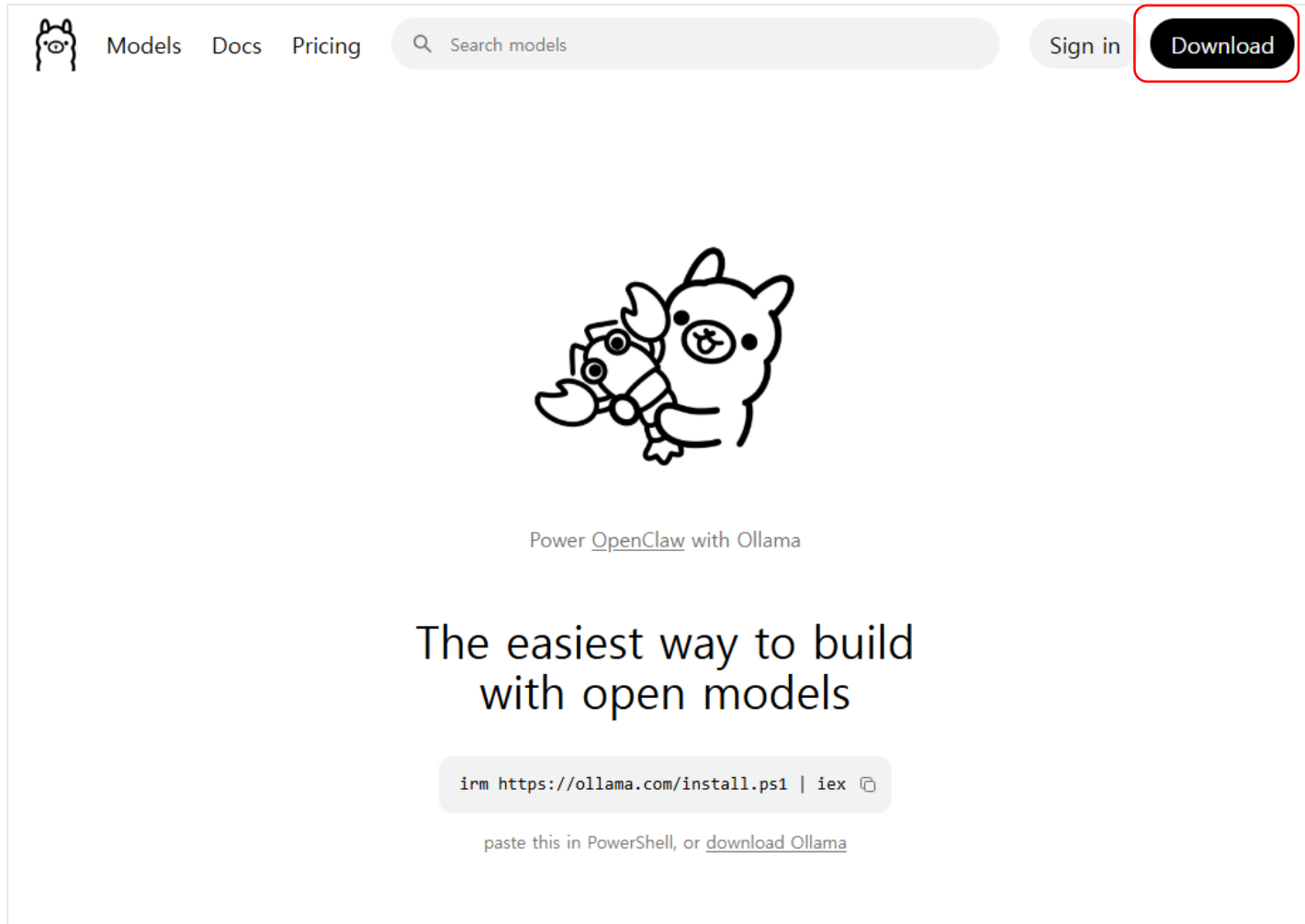
1. 딥시크 모델

- (실습)올라마와 딥시크-R1 모델 설치하기

- 언어 모델을 내 PC에 설치하는 방법은 여러 가지가 있음
- 허깅페이스에서 언어 모델을 내려 받아 사용할 수도 있고, 메타에서 제공하는 올라마(Ollama)를 이용해서 내려 받을 수도 있음
- 올라마는 인공지능 모델을 효율적으로 배포/실행하는 오픈소스 프레임워크로, 개발자가 쉽고 빠르게 인공지능 모델을 활용할 수 있도록 돕는 도구
- 이 플랫폼은 특히 로컬 환경에서 언어 모델을 쉽게 설치하고 실행하는 데 최적화되어 있음
- 실습은 윈도우 운영 체제를 기본으로 올라마를 활용해 딥시크-R1 모델을 내 PC에 설치해 보겠음

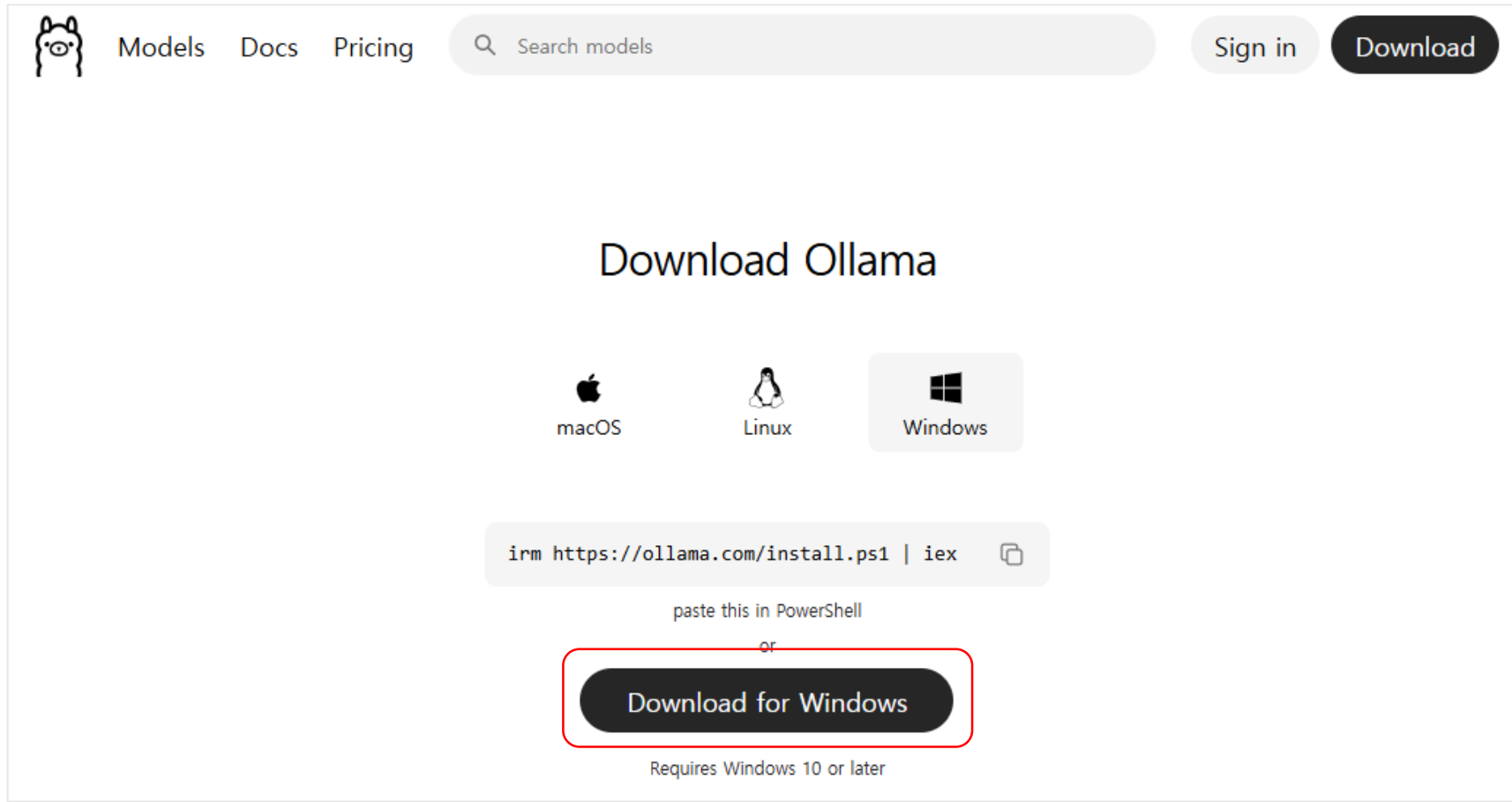
1. 딥시크 모델

- 올라마 웹 사이트(<https://ollama.com>)에 접속하고 [Download] 버튼 클릭



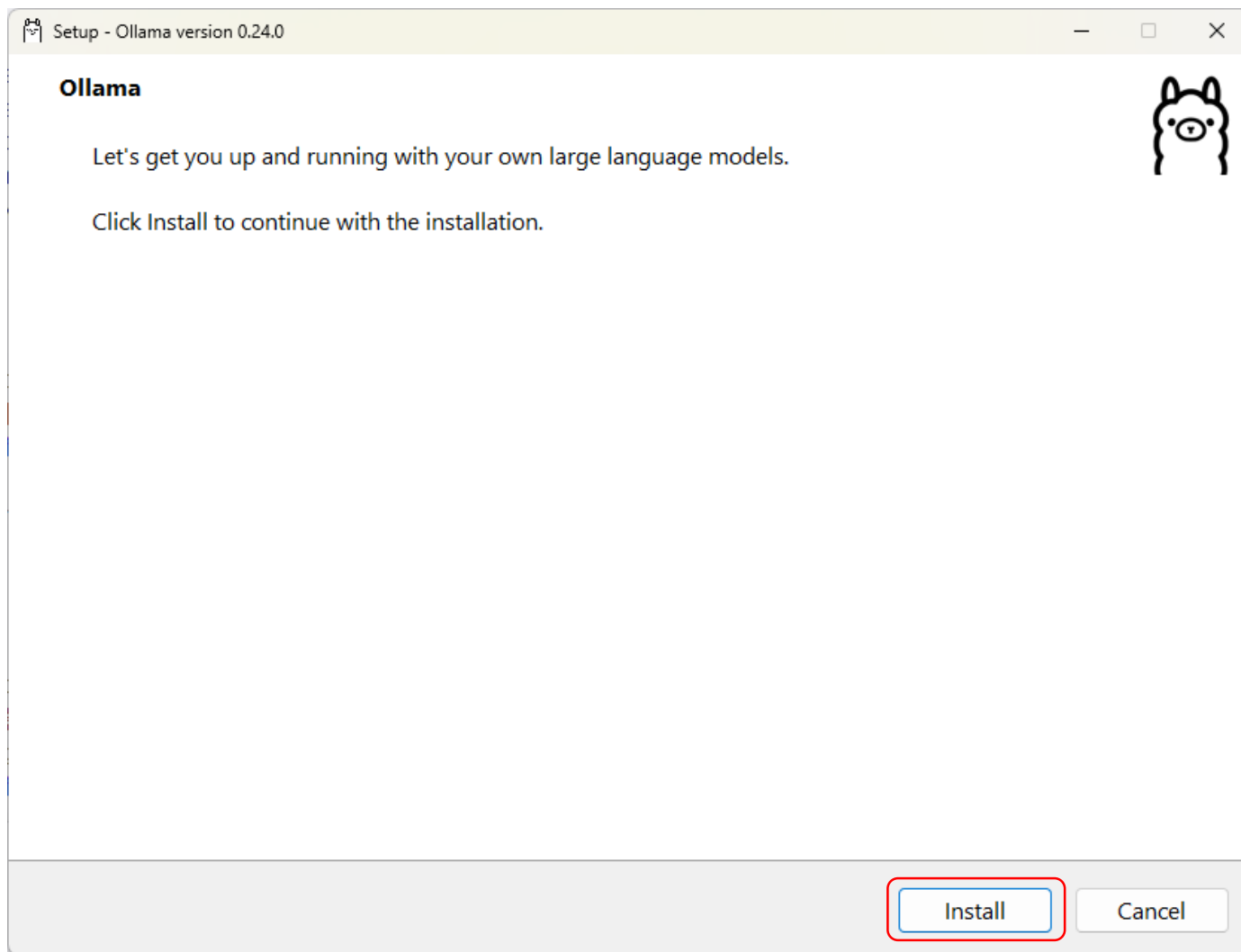
1. 딥시크 모델

- 올라마 웹 사이트(<https://ollama.com>)에 접속하고 [Download] 버튼 클릭



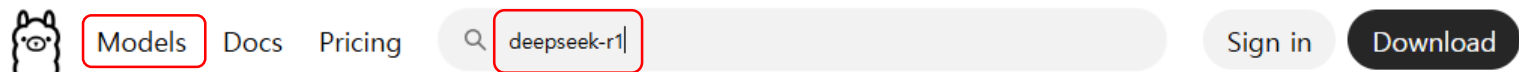
1. 딥시크 모델

- 내려 받은 프로그램을 실행하고 [Install] 버튼을 클릭해 올라마 설치



1. 딥시크 모델

- 올라마 웹 사이트에서 [Models]를 클릭하고 [deepseek-r1] 모델을 선택



Cloud Embedding Vision Tools Thinking Popular ▾

deepseek-r1

DeepSeek-R1 is a family of open reasoning models with performance approaching that of leading models, such as O3 and Gemini 2.5 Pro.

tools thinking 1.5b 7b 8b 14b 32b 70b 671b

↓ 85.6M Pulls ↻ 35 Tags ⌚ Updated 10 months ago

deepscaler

A fine-tuned version of Deepseek-R1-Distilled-Qwen-1.5B that surpasses the performance of OpenAI's o1-preview with just 1.5B parameters on popular math evaluations.

1.5b

↓ 1.2M Pulls ↻ 5 Tags ⌚ Updated 1 year ago

openthinker

A fully open-source family of reasoning models built using a dataset derived by distilling DeepSeek-R1.

7b 32b

↓ 1.1M Pulls ↻ 15 Tags ⌚ Updated 1 year ago

1. 딥시크 모델

- 딥시크-R1 모델은 경량화 정도에 따라 여러 모델을 제공. 자신의 컴퓨터 성능에 따라 선택. 14b 모델 선택하여 진행

Models				View all →
Name	Size / Usage	Context	Input	
deepseek-r1:latest	5.2GB	128K	Text	
deepseek-r1:1.5b	1.1GB	128K	Text	
deepseek-r1:7b	4.7GB	128K	Text	
deepseek-r1:8b latest	5.2GB	128K	Text	
deepseek-r1:14b	9.0GB	128K	Text	
deepseek-r1:32b	20GB	128K	Text	
deepseek-r1:70b	43GB	128K	Text	
deepseek-r1:671b	404GB	160K	Text	

1. 딥시크 모델

- 명령어를 복사해 VS Code 터미널 창에서 power shell 모드에서 설치 진행

deepseek-r1:14b

↓ 85.6M Downloads ⌚ Updated 10 months ago

DeepSeek-R1 is a family of open reasoning models with performance approaching that of leading models, such as O3 and Gemini 2.5 Pro.

[tools](#) [thinking](#) [1.5b](#) [7b](#) [8b](#) [14b](#) [32b](#) [70b](#) [671b](#)

CLI [cURL](#) [Python](#) [JavaScript](#)

```
ollama run deepseek-r1:14b
```



Details

Updated 11 months ago

c333b7232bdb · 9.0GB ·

model	arch <code>qwen2</code> · parameters <code>14.8B</code> · quantization <code>Q4_K_M</code>	9.0GB
-------	--	-------

license	MIT License Copyright (c) 2023 DeepSeek Permission is hereby grante...	1.1kB
---------	--	-------

params	{ "stop": ["< begin__sentence >", "< end__sentence >",	148B
--------	---	------

template	{{- if .System }}{{ .System }}{{ end }} {{- range \$i, \$_ := .Messag...	556B
----------	--	------

1. 딥시크 모델

- 만약 다음과 같은 오류가 발생한다면 컴퓨터를 다시 부팅한 후 설치

```
● (.venv) PS C:\ai_agent\src\except_chap05> ollama run deepseek-r1:14b
ollama : 'ollama' 용어가 cmdlet, 함수, 스크립트 파일 또는 실행할 수 있는 프로그램 이름으로 인식되지 않습니다. 이름이 정
확한지 확인하고 경로가 포함된 경우 경로가 올바른지 검증한 다음 다시 시도하십시오.
위치 줄:1 문자:1
+ ollama run deepseek-r1:14b
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (ollama:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
```

1. 딥시크 모델

- 설치가 완료되면 다음과 같은 프롬프트가 나옴

```
(.venv) PS C:\ai_agent\src\except_chap05> ollama run deepseek-r1:14b
```

```
pulling manifest
```

```
pulling 6e9f90f02bb3: 100%
```

```
pulling c5ad996bda6e: 100%
```

```
pulling 6e4c38e1172f: 100%
```

```
pulling f4d24e9138dd: 100%
```

```
pulling 3c24b0c80794: 100%
```

```
verifying sha256 digest
```

```
writing manifest
```

```
success
```

```
>>> Send a message (/? for help)
```

```
9.0 GB
```

```
556 B
```

```
1.1 KB
```

```
148 B
```

```
488 B
```

1. 딥시크 모델

- 터미널 창에서 대화를 바로 시작할 수 있음
- 딥시크-R1은 답변하기 전에 추론 과정을 거치는 모델
- 대답하기 전에 [Thinking...] [...done thinking] 사이에 먼저 생각하는 과정을 거치며 그 과정을 사용자에게 보여 줌
- 대화를 종료하려면 ctrl + D를 누르면 됨
- 대화 내용에서 알 수 있듯이 내 컴퓨터에 설치해서 사용하는 경량화된 언어 모델인데도 답변의 품질이 괜찮음(다음 page 참조)

1. 딥시크 모델

● 실행 결과

>>> Hello! Nice to meet you.

Thinking...

Alright, someone just said "Hello! Nice to meet you." I should respond warmly.

I want to make them feel welcome and show interest in getting to know them better.

Maybe I can ask how they're doing or if there's something specific they'd like to talk about.

Keeping it friendly and open-ended seems like the best approach.

...done thinking.

Hello! It's nice to meet you too. How are you doing? If you have any questions or need help with something, feel free to let me know—I'm here to assist you!

>>> 한국어도 하실 수 있나요? 할 수 있다면 한국어로 대화해요.

Thinking...

Alright, so the user initially greeted me in English and I responded warmly, asking how they were doing. Then, they switched to Korean with "한국어도 하실 수 있나요?" which means "Can you also speak Korean?" They then added a condition: "할 수 있다면 한국어로 대화해요," meaning "If you can, let's chat in Korean."

I need to figure out how to respond appropriately. First, I should acknowledge their request and confirm that I do understand Korean. It's important to be polite and friendly, just like the initial interaction.

I want to make sure they feel comfortable continuing the conversation in Korean, so offering assistance in their preferred language is key. Maybe I can ask them how I can help or if there's something specific they need assistance with.

Also, since they switched languages mid-conversation, I should be prepared for further dialogue in Korean without any issues. Ensuring a smooth transition will make the interaction seamless for them.

So, my response should be positive, affirming that I understand and am ready to assist in Korean. Maybe something like, "네, 한국어로도 도와드리겠습니다! 어떻게 도와드릴까요?" which means "Yes, I can help in Korean too! How can I assist you?"

I think this approach maintains the friendly tone while clearly offering my assistance in their preferred language.

...done thinking.

네, 한국어로도 도와드리겠습니다! 어떻게 도와드릴까요?

>>> Send a message (/? for help)

2. 랭체인에서 딥시크 모델 사용하기

- 랭체인을 활용해 개발하면 언어 모델을 간단히 교체할 수 있음
- 이번 절에서는 랭체인과 딥시크를 활용해 간단한 챗봇을 만들어 보겠음

- (실습) 딥시크와 랭체인으로 챗봇 만들기

- ChatOllama를 사용하기 위해 패키지 설치

```
(.venv)
```

```
$ pip install langchain-ollama
```


2. 랭체인에서 딥시크 모델 사용하기

- (실습) 딥시크와 랭체인으로 챗봇 만들기
 - deepseek_simple_chatbot.py 파일 생성하고 다음 코드 작성

```
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage, AIMessage, SystemMessage

llm = ChatOllama(model="deepseek-r1:14b") # 만약 컴퓨터 성능이 부족하다면 7b, 8b 모델 선택

messages = [
    SystemMessage("너는 사용자를 도와주는 상담사야."),
]

while True:
    user_input = input("사용자: ")

    if user_input == "exit":
        break

    messages.append( HumanMessage(user_input) )

    response = llm.stream(messages)

    ai_message = None
    for chunk in response:
        print(chunk.content, end="")
        if ai_message is None:
            ai_message = chunk
        else:
            ai_message += chunk
    print('')

    messages.append(AIMessage(ai_message.content))
```

2. 랭체인에서 딥시크 모델 사용하기

● (실습) 딥시크와 랭체인으로 챗봇 만들기

• 실행 결과

사용자: 마일즈 데이비스에 대해 한글로 알려줘.

마일즈 데이비스(Miles Davis)는 20세기 미국의 재즈 트럼펫티스트로, jazz 역사상 가장 영향력 있는 아티스트 중 한 명입니다. 그는 1940년대부터 활동을 시작해, 50년대와 60년대에 걸쳐 여러 스타일의 음악을 통해 재즈의 경계를 확장시켰습니다.

주요 이력

- ****생애****: 1926년 1월 26일 일리노이주 디카터에서 태어나 1991년 9월 28일 로스앤젤러스에서 사망했습니다.
- ****음악적 경력****: 그는 청년기부터 재즈 거물 딜리지 Gillespie 밑에서 연주하며 이름을 올렸고, 나중에는 자신의 밴드를 결성해 커리어를 이어갔습니다. 또한, 몽크(Thelonious Monk)와의 작업도 유명합니다.

음악적 스타일

데이비스는 다양한 스타일을 실험했는데, 대표적인 것으로는:

1. ****쿨 재즈(Cool Jazz)****: 40~50년대에 전 세계적으로 큰 인기를 끌었던 스타일로, 그의 앨범 "생의 출생(Birth of the Cool)"은 이 장르의 대표작으로 꼽힙니다.
2. ****하드 보프(Hard Bop)****: 50년대 후반부터 60년대까지 유행한 스타일로, 블루스와 휴고 모리슨(Count Basie)의 영향을 받은 음악입니다.
3. ****퓨전 재즈(Fusion Jazz)****: 70년대에 들어서면서 실험적인 접근으로 일렉트릭 기타와 기타樂기를 사용한 음악을 탐구했습니다.

유명 앨범

- "생의 출생(Birth of the Cool)" (1952)
- "블루스 넘버즈(Kind of Blue)" (1959) – 빅밴드 재즈 앨범 중 가장 많이 팔린 작품 중 하나입니다.
- "인 어 플레이스(In a Silent Way)" (1969)
- "록스타 스토리(Rock Story)" (1973)

영향력

데이비스는 재즈뿐만 아니라搖滾, 힙합等多种 음악 장르에도 큰 영향을 미쳤습니다. 그의 즉흥연주와 창의적인 접근은 후대의 많은 아티스트들에게 모티브가 되었습니다.

추천 음악

- "고(Go)" feat. 쿠퍼런스(Buddy Rich, John McLaughlin)
- "위드 4 웨이즈(With Four)웨인카츠(Johnny재즈) & 채리스(Charlie Mingus)"
- "블루스 넘버즈(Kind of Blue)"

마일즈 데이비스는 그의 창의성과 혁신적인 음악으로 재즈 역사에 불멸의 업적을 남겼습니다. 그의 음악은 여전히 많은 사랑을 받고 있습니다.

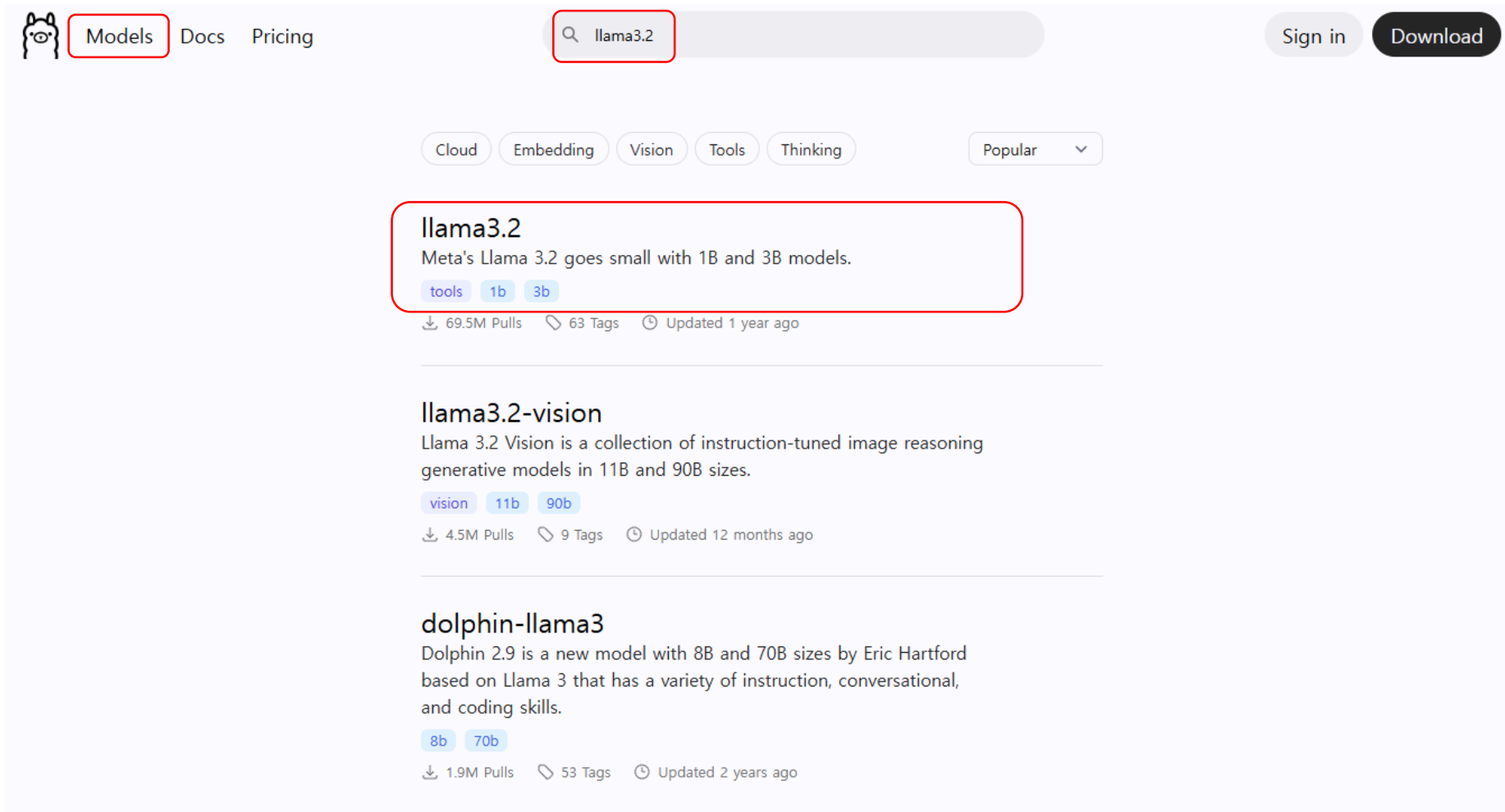
사용자: exit

3. 로컬에서 라마와 임베딩 모델 사용하기

- 로컬 컴퓨터에서 챗봇이나 RAG 기반 프로그램을 개발하는 방법을 알아봄
- 메타는 API로 활용할 수 있는 대규모 언어 모델과 로컬에서 구동할 수 있는 소규모 언어 모델을 모두 제공
- 여기서는 소규모 언어 모델로 자주 활용되는 메타의 라마 모델을 사용함

3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 메타의 라마 모델을 로컬에서 구동하기
 - 올라마 웹 사이트에서 [Models]를 클릭하고 [llama3.2] 모델을 선택



3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 메타의 라마 모델을 로컬에서 구동하기
 - llama3.2의 3b 모델을 사용. 여기서 3b는 이 모델의 매개변수
 - 같은 모델이라도 매개변수가 많을수록 성능이 좋고 계산 자원이 많이 필요
 - llama3.2의 3b 모델은 2GB로 용량이 비교적 작아서 개인 컴퓨터에서 구동할 수 있음

Models				View all →
Name	Size / Usage	Context	Input	
llama3.2:latest	2.0GB	128K	Text	
llama3.2:1b	1.3GB	128K	Text	
llama3.2:3b latest	2.0GB	128K	Text	

3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 메타의 라마 모델을 로컬에서 구동하기
 - 명령어를 복사해 VS Code 터미널 창에서 power shell 모드에서 설치 진행

llama3.2:3b

📄 69.5M Downloads ⌚ Updated 1 year ago

Meta's Llama 3.2 goes small with 1B and 3B models.

[tools](#) [1b](#) [3b](#)

[CLI](#) [cURL](#) [Python](#) [JavaScript](#)

```
ollama run llama3.2:3b
```



Details

Updated 1 year ago	a80c4f17acd5 · 2.0GB ·
model	arch: llama - parameters: 3.2B - quantization: Q4_K_M 2.0GB

3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 메타의 라마 모델을 로컬에서 구동하기
 - 설치가 완료되면 다음과 같은 프롬프트가 나옴

```
○ (.venv) PS C:\ai_agent\src\except_chap05> ollama run llama3.2:3b
pulling manifest
pulling dde5aa3fc5ff: 100% 2.0 GB
pulling 966de95ca8a6: 100% 1.4 KB
pulling fcc5a6bec9da: 100% 7.7 KB
pulling a70ff7e570d9: 100% 6.0 KB
pulling 56bb8bd477a5: 100% 96 B
pulling 34bb5ab01051: 100% 561 B
verifying sha256 digest
writing manifest
success
>>> Send a message (/? for help)
```

3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 메타의 라마 모델을 로컬에서 구동하기

- 실행 결과

```
>>> Hello! Nice to meet you.
```

```
Nice to meet you too! I'm happy to chat with you. Is there something on your mind that you'd like to talk about, or are you looking for some conversation starters? I'm all ears (or rather, all text)!
```

```
>>> 나 영어 못하니까 한국어로 설명해줘.
```

```
이제부터 한국어로 이야기할게!
```

```
처음 만난 사람이라서 인사하는 것만으로도 좋지만, 나도 애타도록 만남으로 인하여 감사합니다.
```

```
지금은 어떤 주제에 대해 thảo화 하시겠는지에 好奇心하고 있습니다? 혹시이가 관심 있으신다면, 我와 함께 이야기를 시작할 수 있습니다.
```

```
>>> Send a message (/? for help)
```

- 대화를 종료하려면 ctrl + D를 누르거나, '/bye'라고 입력하면 됨

3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 라마 기반으로 간단한 챗봇 만들기
 - llama_simple_chatbot.py 파일 생성하고 다음 코드 작성

```
# from langchain_openai import ChatOpenAI
from langchain_ollama import ChatOllama
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

# 모델 초기화
# llm = ChatOpenAI(model="gpt-5-nano")
llm = ChatOllama(model="llama3.2:3b")

messages = [
    SystemMessage("You are a helpful assistant."),
]

while True:
    user_input = input("You\t: ").strip()

    if user_input in ["exit", "quit", "q"]:
        print("Goodbye!")
        break

    messages.append(HumanMessage(user_input))

    response = llm.invoke(messages)
    print("Bot\t: ", response.content)

    messages.append(AIMessage(response.content))
```

3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 라마 기반으로 간단한 챗봇 만들기

- 실행 결과

You : Hello. How are you?

Bot : I'm just a language model, so I don't have feelings or emotions like humans do, but I'm functioning properly and ready to help with any questions or tasks you may have! How about you? Is there something specific you'd like assistance with today?

You : 한국어로만 대답해줘

Bot : 안녕하세요. 저는 인공지능 기반의 언어 모델이기 때문에 feelings나 감정을 가지고 있지 않습니다. 하지만 제 functionality가 잘 작동하고, 도움을 원하시면 언제든지 질문이나 task를 받아주시길 바랍니다!

You :

3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 로컬 임베딩 모델 사용하기

- 로컬에서 사용할 수 있는 언어 모델이 있는 것처럼 로컬에서 사용할 수 있는 임베딩 모델도 여러 가지 존재
- 아직 대부분의 모델이 영어로 학습되어 있어서 한국어 문서에 사용할 때 동일한 성능을 보장하기는 어려움
- BAAI/bge-m3 모델을 사용해 보겠음
 - 모든 한국어 문서에서 최적의 성능을 보장하지는 않지만 초보자도 활용하기 쉽고 성능도 괜찮음

3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 로컬 임베딩 모델 사용하기
 - 새로운 주피터 노트북 파일 embedding_bge_m3.ipynb 준비
 - data 폴더를 만들고 사용할 pdf 파일들을 준비

```
from glob import glob

for g in glob('./data/*.pdf'):
    print(g)
```

3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 로컬 임베딩 모델 사용하기

- 청크 분할하기

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

def read_pdf_and_split_text(pdf_path, chunk_size=1000, chunk_overlap=100):
    """
    주어진 PDF 파일을 읽고 텍스트를 분할합니다.
    매개변수:
        pdf_path (str): PDF 파일의 경로.
        chunk_size (int, 선택적): 각 텍스트 청크의 크기. 기본값은 1000입니다.
        chunk_overlap (int, 선택적): 청크 간의 중첩 크기. 기본값은 100입니다.
    반환값:
        list: 분할된 텍스트 청크의 리스트.
    """
    print(f"PDF: {pdf_path} -----")

    pdf_loader = PyPDFLoader(pdf_path)
    data_from_pdf = pdf_loader.load()

    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size, chunk_overlap=chunk_overlap
    )

    splits = text_splitter.split_documents(data_from_pdf)

    print(f"Number of splits: {len(splits)}\n")
    return splits
```

3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 로컬 임베딩 모델 사용하기

- 임베딩 모델 설치

```
(.venv)
```

```
$ pip install langchain-huggingface
```

- 파이토치 설치하기(option)

- CUDA를 지원하는 NVIDIA 그래픽 카드를 갖고 있다면 파이토치를 설치

```
(.venv)
```

```
$ pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124
```

3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 로컬 임베딩 모델 사용하기

- 임베딩 모델 선언

```
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="BAAI/bge-m3",
    model_kwargs = {'device': 'cpu'}, # cuda 지원하면 cuda, 아닌 경우 cpu
    encode_kwargs = {'normalize_embeddings': True}, # 임베딩 정규화
)
```

- 임베딩 모델 테스트

```
embeddings.embed_documents("안녕하세요")
```

3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 로컬 임베딩 모델 사용하기
 - pdf 파일 읽고 텍스트 임베딩하기

```
from langchain_chroma import Chroma

import os
persist_directory='./chroma_store'

if os.path.exists(persist_directory):
    print("Loading existing Chroma store")
    vectorstore = Chroma(
        persist_directory=persist_directory,
        embedding_function=embeddings
    )
else:
    print("Creating new Chroma store")

    all_splits = []
    for g in glob('./data/*.pdf'):
        all_splits.extend(read_pdf_and_split_text(g))

    print(f"Total number of splits: {len(all_splits)}")

    vectorstore = Chroma.from_documents(
        documents=all_splits,
        embedding=embeddings,
        persist_directory=persist_directory
    )
```


3. 로컬에서 라마와 임베딩 모델 사용하기

- (실습) 로컬 임베딩 모델 사용하기

- 리트리버 테스트

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})

chunks = retriever.invoke("서울시 쓰레기 저감 정책")

for chunk in chunks:
    print(chunk.metadata)
    print(chunk.page_content)
```